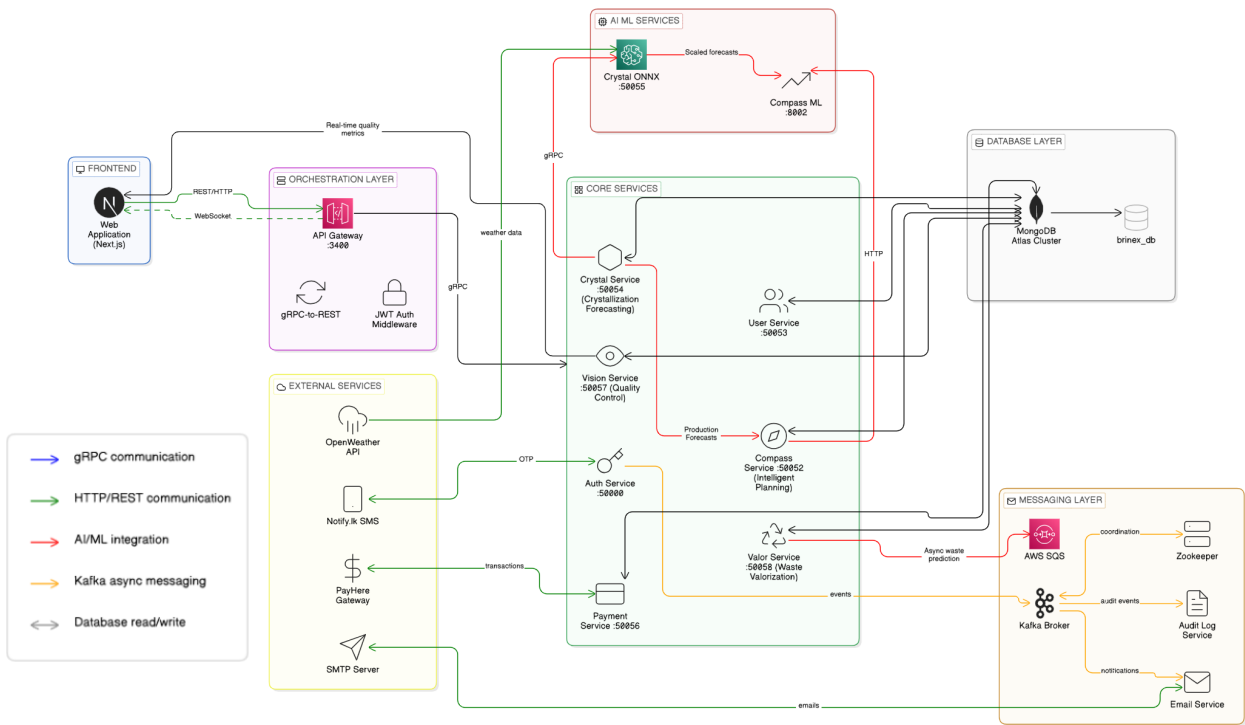


BrineX -System Maturity Report

1. All Four Components Integrated

BrineX is a fully integrated, event-driven microservices platform comprising 11 services communicating across a unified API Gateway. All four AI components are deployed, interconnected, and operational within the same Docker Compose environment on a shared app-network bridge, with 26+ MongoDB Atlas collections across all services.

Integration Architecture:



Cross-Component Integration Points:

From	To	Data	Protocol
API Gateway	Core Services	JWT-authenticated requests	gRPC
Frontend	API Gateway	User requests	REST/HTTP
Crystal ONNX Service (:50055)	Compass ML Service (:8002)	Landowner-scaled production forecasts (bags/month)	gRPC → HTTP
OpenWeatherMap	Crystal ONNX Service	Weather forecast (temp, rain, humidity)	REST
Crystal	Valor	Production volume data (kg/month)	gRPC (planned)
Vision Service	Frontend	Live detection frames + purity metrics	WebSocket
Compass ML Service	Compass Service	Price prediction + demand estimate	HTTP

4 Frontend Modules deployed: Crystal (salt production monitoring), Compass (land and commodity trading), Vision (AI-powered salt detection), Valor (risk and analytics).

2. Major Functionalities Completed

A. Crystal - Crystallization Forecasting

Feature	Status	Description
Daily measurement capture	Complete	8 crystallization parameters logged per day with OpenWeatherMap enrichment
LSTM inference via ONNX Runtime	Complete	30-day sliding window → 480 output values (60 days × 8 parameters)
Monthly production forecasting	Complete	Linear Regression calibration from LSTM output - month+1 and month+2
Seasonal forecasting	Complete	Maha/Yala seasonal breakdown with 95% prediction intervals
Landowner-wise scaling	Complete	Facility forecast scaled by individual bed count ratio (against 7,500 total beds)
Historical weather fallback	Complete	Months 2–12 use 36-month historical averages from calibration_constants.json
Production forecast → Compass	Complete	Month+1 and month+2 forecasts passed directly to Compass service
6-month and 12-month dashboard	Complete	Visual charts for operational and strategic planning

B. Compass -Intelligent Planning Engine

Feature	Status	Description
Harvest plan management	Complete	Full CRUD for landowner harvest plans
Demand forecasting	Complete	Yield ratio algorithm -production forecast × seasonal ratio (Yala: 8.74%, Maha: 10.27%)
Price forecasting	Complete	SARIMAX price model -MAPE 1.16% month+1, 1.35% month+2
Distributor offer management	Complete	Offers created, listed, and matched against landowner production
Seller recommendations	Complete	Ranked by highest price per bag and total bag count -fully auditable
Deal lifecycle management	Complete	Draft → Pending → Accepted → Closed → Cancelled (5 states)
Crystal forecast integration	Complete	Landowner sensor readings drive demand estimate via Crystal production forecast

C. Vision -Computer Vision Quality Control

Feature	Status	Description
YOLOv8 real-time detection	Complete	3-class detection (Pure / Impure / Unwanted) at ~45ms/frame

Purity percentage calculation	Complete	$(\text{pureCount} / (\text{pureCount} + \text{impureCount})) \times 100$ per frame, batch, session
HSV whiteness scoring	Complete	$V \times (1 - S) \times 100$ per pixel -calibratable, independent of human judgment
Weighted quality scoring	Complete	$\times 1.0$ pure, $\times 0.3$ impure, $\times 0$ unwanted -aggregated per crystal and batch
ROI-based filtering	Complete	Targeted analysis of specific camera regions
Batch management	Complete	Batch logging with timestamps -full quality traceability per production run
Real-time WebSocket streaming	Complete	Live detection results delivered to frontend with sub-second latency
24-hour temporal analytics	Complete	Hourly bins for trend analysis across production day
Laboratory plan gate	Complete	3-layer guard stack -JWT → Roles → Plan -restricts Vision to Lab subscribers

D. Valor -Federated Waste Valorization

Feature	Status	Description
Federated Learning training loop	Complete	End-to-end operational -ROUND_START → local training → weight submission → FedAvg aggregation → MODEL_BROADCAST
AWS SQS job queue	Complete	Server + client FIFO queues, Dead Letter Queue, 6 event types supported
Waste prediction system	Complete	8 outputs per inference -4 solid waste, 1 liquid, 3 valorization products
Waste forecast service	Complete	Daily forecasting -auto-fetches weather from dailymeasurements collection
Job management system	Complete	PENDING → PROCESSING → COMPLETED/FAILED lifecycle, full CRUD
Model update service	Complete	S3-based model download and hot-swap via MODEL/UPDATE events

3. Performance Tested

Performance testing was conducted on March 7–8, 2026 using Grafana k6 across five core microservices. Three test scenarios were executed: Smoke (1 VU, ~30 seconds), Average Load (5 VUs, ~2 minutes), and Stress (15 VUs, ~3.5 minutes).

3.1 Overall Results Summary

Service	Smoke	Avg Load	Stress	Peak RPS	Production Readiness
User Service	PASS	PASS	PASS	9.14/s	Ready with Monitoring
Compass Service	PASS	PASS	WARN	3.32/s	Not Ready - Optimize Now
Crystallization Service	PASS	PASS	PASS	14.43/s	Production Ready

Vision Service	PASS	WARN	FAIL	1.27/s	Not Ready - Major Work Required
Waste Valorization Service	PASS	PASS	PASS	15.59/s	Production Ready

3.2 Crystallization Service - Best Overall Performer

Test	Avg (ms)	Med (ms)	P95 (ms)	Max (ms)	Req/s	Checks
Smoke (1 VU)	114.76	84.00	154.42	886.08	1.475	100%
Average Load (5 VUs)	92.15	76.39	153.24	693.16	6.019	100%
Stress (15 VUs)	88.64	76.65	146.66	703.34	14.43	100%

The Crystallization Service is the standout performer across all tests. Average response time actually improves from 114ms (smoke) to 88ms (stress), demonstrating exceptional scaling behavior. 100% check rates were maintained across all three test types with zero threshold failures. At peak stress the service processed 3,112 requests at 14.43 req/s - the highest throughput of any service tested. All five endpoints (salt production, model performance, daily measurement, predicted daily, predicted monthly) remained well within acceptable response times at 15 VUs. The service is immediately production-ready and serves as a reference implementation for the platform.

3.3 Waste Valorization Service - Fastest Response Times

Test	Avg (ms)	Med (ms)	P95 (ms)	Max (ms)	Req/s	Checks
Smoke (1 VU)	3.56	3.42	5.66	6.54	1.655	100%
Average Load (5 VUs)	3.60	3.37	4.91	28.28	6.457	100%
Stress (15 VUs)	11.01	4.32	47.09	335.23	15.594	100%

The Waste Valorization Service delivered the fastest response times of all services tested - averaging 3.56ms under smoke and maintaining sub-15ms averages even at 15 VUs. 100% check rates were recorded across all three test types with zero failures. The stress test processed 3,285 requests at 15.59 req/s - the highest request rate of the entire test suite. The maximum observed response time of 335ms under stress is the lowest ceiling of all services tested.

3.4 User Service - Stable with Minor Tuning Required

Test	Avg (ms)	Med (ms)	P95 (ms)	Max (ms)	Req/s	Checks
Smoke (1 VU)	299.31	178.83	576.71	732.74	1.034	100%
Average Load (5 VUs)	235.22	165.68	413.92	456.48	1.086	100%
Stress (15 VUs)	374.74	226.09	759.86	11,811.39	9.137	98.71%

The User Service maintained stable operation across all test types. The stress test generated 1,938 requests at 9.14 req/s with a 98.71% check rate. The primary concern is the Get All Users endpoint which spiked to 11,811ms maximum under stress, indicating potential database connection pool exhaustion at peak concurrency. The 33% HTTP failure rate across tests reflects expected authentication rejections in the test scenario rather than true service errors, and falls within the defined <50% threshold. Recommended action: add pagination or Redis caching to the Get All Users endpoint.

3.5 Compass Service - Requires Optimization Before Production

Test	Avg (ms)	Med (ms)	P95 (ms)	Max (ms)	Req/s	Checks
Smoke (1 VU)	212.78	4.87	1,020.53	2,287.53	1.309	98.75%
Average Load (5 VUs)	1,160.61	4.19	1,418.51	60,000.29	2.12	98.11%
Stress (15 VUs)	2,376.35	4.62	12,796.64	60,001.26	3.317	93.45%

The Compass Service shows critical performance degradation under concurrent load. Average response time escalates from 213ms (smoke) to 1,161ms (average load) to 2,376ms (stress). Maximum response times hit the 60-second timeout boundary during both average and stress tests, primarily on the Deals and Offers endpoints. The check rate degrades from 98.75% (smoke) to 93.45% (stress), with 93 failed checks under stress - the highest failure count of any service. The Compass Offers endpoint recorded a 2,498ms average and 60,000ms maximum under stress. The service is not production-ready in its current state. Immediate database query optimization, connection pooling, response caching for read-heavy endpoints, and circuit breakers are required.

3.6 Vision Service - Requires Significant Work Before Production

Test	Avg (ms)	Med (ms)	P95 (ms)	Max (ms)	Req/s	Checks
Smoke (1 VU)	468.64	340.81	1,151.12	1,350.95	0.961	100%
Average Load (5 VUs)	3,151.37	1,439.94	12,343.61	36,436.28	1.113	89.17%
Stress (15 VUs)	6,534.54	1,712.39	25,970.79	60,003.15	1.267	84.17%

The Vision Service handles single-VU load adequately but degrades severely under concurrent access - a 14× increase in average response time from smoke (469ms) to stress (6,535ms). The Vision Stats endpoint is the most resource-intensive, reaching 8,700ms average and 60,003ms maximum under stress (timeout boundary reached). The Vision Detections endpoint averaged 7,135ms under stress, reflecting the heavy ML compute overhead of concurrent YOLOv8 inference. The health endpoint consistently responded in under 10ms across all tests, confirming the service process is alive but compute-bound. The check rate fell to 84.17% under stress with 85 failed checks. The service is not production-ready for concurrent access. Asynchronous processing via a job queue, ONNX model quantization, and horizontal scaling across multiple inference workers are required.

3.7 Priority Action Plan

Priority	Service	Action Required	Impact
P1	Compass Service	Optimize Deals/Offer DB queries; add connection pooling, caching, circuit breakers	Critical - eliminates 60s timeouts
P1	Vision Service	Implement async job queue for Detections/Stats; explore ONNX quantization and GPU acceleration	Critical - eliminates timeout failures under concurrent load
P2	User Service	Add Redis caching or pagination to Get All Users endpoint	High - prevents stress spikes
P3	Waste Valorization	Add http_req_failed thresholds to test scripts; validate non-mock data in test environment	Medium - test completeness
P3	Crystallization	Continue monitoring; no immediate action required	Low - already optimal

4. Risks Identified and Mitigated

System-Wide Risks

Risk	Mitigation	Status
Single point of failure at API Gateway	All 11 services independently containerized with separate CI/CD pipelines -each can be redeployed independently	Mitigated
JWT token compromise	3-layer guard stack (JWT → Roles → Plan) applied globally -OTP-based auth with Notify.lk SMS	Mitigated
Data schema drift across services	11 gRPC proto files enforce type-safe contracts -breaking changes caught at compile time	Mitigated
Audit trail gaps	All HTTP requests and service events logged asynchronously via Kafka to MongoDB (6 compound indexes, 16 fields per event) -zero latency impact	Mitigated
Blocking async operations	Email and audit logging decoupled via Kafka -fire-and-forget, no user-facing request blocked	Mitigated
Multi-tenancy data leakage	8 user roles with plan-based access control -collections isolated per service, no shared schemas	Mitigated

Component-Specific Risks

Component	Risk	Mitigation	Status
Crystal	Weather API unavailability for months 2–12 forecast	Historical monthly averages (36-month window) stored in calibration_constants.json as fallback	Mitigated
Crystal	LSTM production error propagating to Compass demand estimate	±15% confidence bounds surfaced to landowner -demand presented as estimate, not guarantee	Mitigated
Crystal	Farm shutdown months (cyclones, equipment failure) skewing yield ratios	Median yield ratio used instead of mean -robust to near-zero outlier months	Mitigated
Compass	ARIMA/Holt-Winters failing when farm adds new beds	Linear Regression uses causal inputs (beds, rain, temp, seasonality) -new beds immediately reflected in forecast	Mitigated
Compass	Price direction prediction unreliable	SARIMAX optimized for price level accuracy (MAPE 1.16%) not direction -mean-reverting market behavior documented	Mitigated
Vision	Unauthorized access to quality data	Laboratory plan gate -3-layer guard stack prevents non-Lab users from accessing Vision module	Mitigated
Vision	False positives from overlapping detections	NMS deduplication at IoU threshold 0.45 -reduces false positives while preserving multi-class detections	Mitigated

Valor	Cold start -no model on first deployment	seed_initial_model.py pre-seeds MongoDB; clients fall back to local .pkl if MongoDB unavailable	Mitigated
Valor	Privacy compliance - raw data exposure	No raw data transmitted -only model weights via FedAvg aggregation; SQS FIFO encrypted in transit	Mitigated
Valor	Model drift over time	FL server triggers on-demand training rounds; round metrics (R^2 , MAE per client) tracked in fl_rounds collection	Mitigated
Valor	Client connectivity loss during training	SQS long-polling tolerates intermittent connectivity; clients complete local training and submit when reconnected; 4-day message retention	Mitigated
Valor	AWS cost overrun	Free tier deployment (t2.micro); FL server can be stopped between training rounds (desired_count=0)	Mitigated
Valor	SQS message race conditions	MessageGroupId per round (fl-{model_id}-round-{N}); server-side _running flag prevents concurrent sessions	Mitigated

4. Standards and Best Practices Applied

API Design and Communication

- RESTful API design with Swagger documentation at /api/v1
- gRPC with Protocol Buffers -11 proto files define all cross-service contracts
- Type-safe request/response schemas enforced at compile time across all 11 services
- Bearer JWT authentication on all protected endpoints
- WebSocket for real-time data streams (Vision detection, live dashboard updates)
- Kafka async messaging for event-driven decoupling - email notifications and audit logging handled as non-blocking background consumers via dedicated Kafka topics

Security

- Defense in depth: JWT → Role → Subscription Plan guard stack applied globally at API Gateway
- OTP-based authentication via Notify.lk SMS for landowner onboarding
- PayHere payment gateway integration with server-side verification
- AWS SQS FIFO queues with encryption in transit for Valor event coordination
- Zero raw data transmission in federated learning -only model weights shared

ML and AI Engineering

- ONNX Runtime for all inference -native Node.js, no Python subprocess, microsecond latency after load
- Model versioning via MODEL_VERSION environment variable -tracked in fl_models collection

- Rolling-origin cross-validation for SARIMAX price model -leakage-free accuracy reporting
- Median yield ratio over mean -robust to farm shutdown outlier months
- Causal model (Linear Regression) over autoregressive alternatives (ARIMA, Holt-Winters) -new farm conditions immediately reflected
- Ensemble model for Valor -Neural Network + XGBoost + tree ensembles + Ridge regression meta-learner -validation R^2 -weighted aggregation

Data Management

- MongoDB Atlas with isolated collections per service -no cross-service schema coupling
- 26+ collections with compound indexes for query performance
- Kafka-based audit logging -16 fields per event, 6 compound indexes, fully non-blocking
- Calibration constants stored in JSON -historical weather averages, regression coefficients, yield ratios

DevOps and Deployment

- Docker Compose orchestration -all 11 services containerized with independent Dockerfiles
- 11 CI/CD pipelines (GitHub Actions) -path-filtered per service, no unnecessary rebuilds
- app-network bridge -internal service discovery without public exposure
- Dead Letter Queue (DLQ) for failed SQS messages -no silent failure on Valor job processing
- Environment-variable driven configuration -no hardcoded credentials or endpoints

Frontend and UX

- Next.js 14 with server-side rendering
- shadcn/ui component library -consistent design system across all 4 modules
- Internationalization -Sinhala, Tamil, English via next-intl
- Mobile-responsive layout
- Role-based UI rendering -components conditionally rendered based on subscription plan